ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

BÀI BÁO CÁO

Học phần: THỰC HÀNH PHÁT TRIỂN HỆ THỐNG TRÍ TUỆ NHÂN TẠO

Chủ đề:

Hệ thống nhận diện biển số xe trong thời gian thực - Kiểm tra thông tin chủ xe.

MSSV	Họ tên	Vai trò
24022276	Lê Mạnh Cường	Backend / DevOps
24022376	Trần Tuấn Kiệt	AI Engine + Model versioning
24022294	Nguyễn Minh Đức	Frontend

Mục lục

1 Giới thiệu **1**
1.1 Bối cảnh và động lực 1
1.2 Mục tiêu đề tài 1
1.3 Phạm vi 1

2 Phân tích yêu cầu **1**
2.1 Tác nhân (Actors) 1
2.2 Yêu cầu chức năng 2
2.3 Yêu cầu phi chức năng 3

3 Kiến trúc phần mềm **3**
3.1 Sơ đồ kiến trúc tổng thể 3
3.2 Phân tách trách nhiệm các service 4
3.3 Nguyên tắc thiết kế 4
3.4 Tech stack 5

4 Thiết kế chi tiết **5**
4.1 Mô hình AI – pipeline LPR 5
4.2 Cơ sở dữ liệu 5
4.3 Luồng xử lý nhận diện không đồng bộ 6
4.4 API tóm tắt 6
4.5 Quản lý phiên bản model & dữ liệu 6

5 Triển khai **6**
5.1 Khởi động nhanh 6
5.2 Workflow developer 6

6 Giao diện người dùng **7**
6.1 Đăng nhập 7
6.2 Bảng giám sát (Dashboard) – dành cho Admin 7
6.3 Giám sát trực tuyến (Live view) 8
6.4 Lịch sử ra vào (History logs) 8
6.5 Tra cứu phương tiện 9
6.6 Quản lý xe & bulk-import (Admin) 10
6.7 Cảnh báo BLACKLIST realtime 10

7 Phân công công việc **11**
7.1 Bảng phân công chi tiết 12
7.2 Tỷ lệ đóng góp tổng thể 12

8 Kết luận **13**

NỘI DUNG

1 Giới thiệu

1.1 Bối cảnh và động lực

Tại các bãi đỗ xe, khu chung cư, trụ sở cơ quan, việc kiểm soát phương tiện ra vào hiện vẫn chủ yếu thực hiện thủ công: bảo vệ ghi chép biển số, đối chiếu danh sách giấy hoặc gọi điện xác minh. Cách làm này tốn nhân lực, chậm trễ và không có dữ liệu vết để truy vết sau sự cố. Khi xảy ra trường hợp xe vi phạm, xe trong danh sách cấm hoặc xe nghi vấn, hệ thống thủ công thường không kịp cảnh báo cho người có trách nhiệm.

Trong khi đó, các camera giám sát IP (RTSP) đã được triển khai rộng rãi. Việc kết hợp camera sẵn có với mô hình thị giác máy tính để *nhận diện biển số tự động* (Automatic License Plate Recognition - ALPR) và đối chiếu với cơ sở dữ liệu chủ xe là hướng đi khả thi, chi phí thấp, đem lại giá trị trực tiếp cho người vận hành.

1.2 Mục tiêu đề tài

Nhóm xây dựng **OwnerCheckAI** - một hệ thống đầu cuối, đáp ứng các mục tiêu sau:

- Đọc luồng video RTSP từ nhiều camera *thời gian thực*; phát hiện biển số và nhận dạng ký tự với độ chính xác cao.
- Đối chiếu biển số nhận được với CSDL chủ xe, trả về thông tin (chủ sở hữu, hãng, loại xe, màu).
- Đối chiếu với danh sách theo dõi (`BLACKLIST` / `WHITELIST`); khi phát hiện xe `BLACKLIST` thì *đẩy cảnh báo realtime* cho quản trị viên qua Server-Sent Events.
- Lưu *log nhận diện* kèm ảnh crop biển số làm bằng chứng.
- Hỗ trợ *tìm xe tương đồng theo ảnh* (image similarity search) để cứu các trường hợp OCR sai biển.
- Quản lý vận hành: CRUD camera, xe, người dùng; dashboard thống kê; bulk-import dữ liệu xe; manual verify khi OCR thiếu chính xác.

1.3 Phạm vi

Phạm vi đồ án giới hạn ở mức demo end-to-end chạy trên một máy tính bằng `docker compose`: 10 container, có thể giả lập camera bằng MediaMTX phát lại file MP4 thành luồng RTSP - không yêu cầu camera vật lý khi nghiệm thu. Phần triển khai Kubernetes, CI/CD và monitoring không nằm trong phạm vi.

2 Phân tích yêu cầu

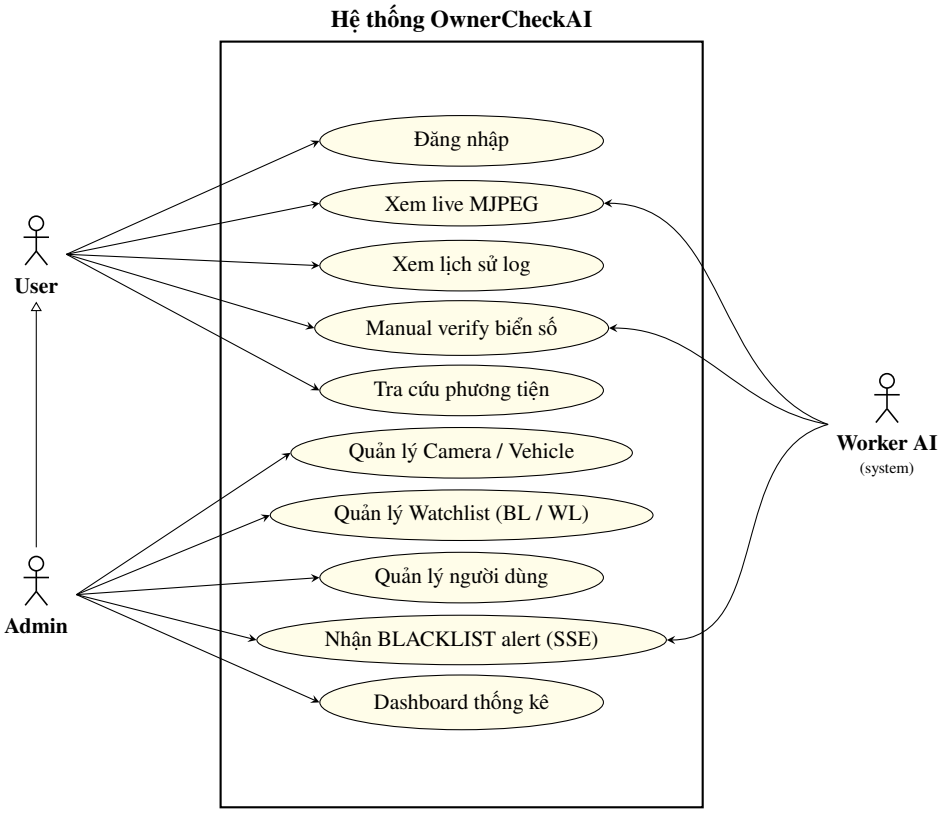
2.1 Tác nhân (Actors)

Hệ thống có hai vai trò chính:

- **User** (Nhân viên trực ca / Monitor staff): xem live MJPEG, lịch sử log, thực hiện manual verify, tra cứu xe theo ảnh.
- **Admin**: tất cả quyền của User *cộng với* quản lý user / camera / vehicle / watchlist, bulk-import, xem dashboard thống kê, nhận realtime BLACKLIST alert.

Ngoài ra hệ thống có hai *tác nhân hệ thống* (system actors) chạy ngầm:

- **Camera RTSP**: nguồn dữ liệu đầu vào.
- **Worker AI**: tiến trình tiêu thụ hàng đợi để chạy pipeline YOLO + PaddleOCR.



Hình 1: Use case diagram của OwnerCheckAI

2.2 Yêu cầu chức năng

1. Đăng nhập / đăng ký bằng JWT; bootstrap: user đầu tiên tự động được gán role `ADMIN`.
2. Phát hiện biển số trong từng frame video; nhận dạng ký tự; đối chiếu chủ xe; kiểm tra watchlist.
3. Streaming MJPEG cho frontend xem trực tiếp, độc lập với pipeline AI để không ảnh hưởng frame rate hiển thị.
4. Đẩy event `blacklist_match` qua SSE cho admin đang mở trang.
5. Lưu log nhận diện + ảnh crop; hỗ trợ filter / sort / verify.
6. Bulk-import vehicles từ JSON (tối đa 1000 dòng / request).

7. Vector search xe tương đồng theo ảnh: dùng ResNet18 sinh embedding 512-d, lưu vào Qdrant, tìm theo cosine distance.
8. Dashboard: thống kê theo ngày, top biển nhận diện, hoạt động camera.

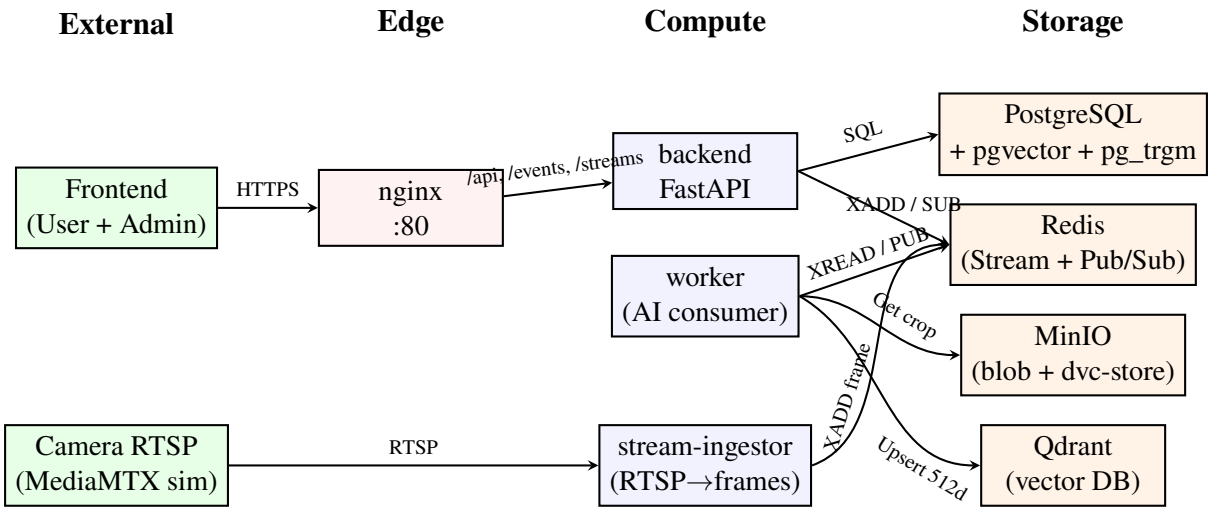
2.3 Yêu cầu phi chức năng

- **Hiệu năng**: API trả phản hồi < 100 ms (xử lý AI chạy nền qua Redis Stream, client polling).
- **Khả năng mở rộng**: backend và worker stateless, có thể scale ngang; toàn bộ trạng thái nằm ở DB / Redis / MinIO / Qdrant.
- **Khả năng tái lập (reproducibility)**: hyperparams tập trung trong `params.yaml`; pipeline train + eval khai báo trong `dvc.yaml`; weight & dataset versioning bằng DVC.
- **Bảo mật**: JWT + bcrypt; mọi port nội bộ bind `127.0.0.1`, chỉ port 80 expose ra ngoài.
- **Khả năng triển khai**: 1 lệnh `docker compose up -d` là chạy được toàn bộ.

3 Kiến trúc phần mềm

3.1 Sơ đồ kiến trúc tổng thể

Hệ thống tổ chức theo 4 lớp: *External* (client + camera), *Edge* (load balancer), *Compute* (các service xử lý không lưu trạng thái) và *Storage* (các hệ lưu trữ có trạng thái).



Hình 2: Sơ đồ kiến trúc phần mềm OwnerCheckAI (4-tier)

3.2 Phân tách trách nhiệm các service

Service	Vai trò	Stateful?
`nginx`	Reverse proxy, route theo path	Không
`frontend`	React SPA build static, nginx-alpine	Không
`backend`	REST API, auth, validation, SSE	Không
`worker`	Consumer Redis Stream, chạy AI inference	Không
`stream-ingestor`	Đọc RTSP, decode, publish frames	Có
`db`	Postgres single instance	Có
`redis`	Stream + pub/sub + LRU cache (512 MB)	Có
`minio`	Blob storage + DVC remote	Có
`qdrant`	Vector DB cho plate embeddings	Có
`mediamtx`	RTSP simulator (file mp4 → RTSP)	—

Bảng 1: Catalog 10 service trong stack

3.3 Nguyên tắc thiết kế

- **Async first** - API trả `202 Accepted` ngay với `job_id`; client poll `GET /recognition/jobs/{id}` để nhận kết quả.
- **Stateless compute** - 3 service `backend`, `worker`, `stream-ingestor` dùng chung 1 image, khác nhau ở `command:` - dễ scale ngang.
- **Backpressure tự nhiên** - Redis Stream làm hàng đợi giữa ingestor và worker; nếu worker chậm, ingestor drop frame (chấp nhận được cho preview).
- **Tách preview khỏi AI** - MJPEG 10 fps độc lập với pipeline AI 2 fps, đảm bảo trải nghiệm xem realtime mượt.
- **Reproducible AI** - `params.yaml` + `dvc.yaml` cho phép rerun đúng stage bị ảnh hưởng khi sửa hyperparam / dataset.

3.4 Tech stack

Lớp	Công nghệ
Backend	FastAPI 0.115, SQLAlchemy 2.0, Pydantic v2
Frontend	React 18 + Vite 5, Ant Design 5, React Query, Recharts
AI Engine	YOLOv8m (Ultralytics), PaddleOCR PP-OCRv3, PyTorch 2.3
Database	PostgreSQL 15 + `pgvector` + `pg_trgm`
Vector DB	Qdrant v1.11 – embedding 512d, Distance.COSINE
Object storage	MinIO (S3-compatible)
Queue / Cache	Redis 7.2 (Streams + Pub/Sub + LRU 512 MB)
Load balancer	nginx 1.26
Stream simulator	MediaMTX (file mp4 $\rightarrow$ RTSP)
Model & data versioning	DVC 3.x + MinIO remote (`dvc-store`)
Container	Docker Compose v2

Bảng 2: Tech stack tổng hợp

4 Thiết kế chi tiết

4.1 Mô hình AI – pipeline LPR

Pipeline xử lý mỗi frame qua 3 bước chính:

1. **Detection** – YOLOv8m fine-tune trên 3.500 ảnh biển số VN, đầu vào 640×640, ngưỡng confidence `det_conf` = 0.5.
2. **Preprocess** – kiểm tra chất lượng crop, deskew, enhance để OCR ổn định hơn ở ánh sáng kém.
3. **Recognition** – PaddleOCR PP-OCRv3 (CTC + SAR MultiHead) trên crop 48×320, ngưỡng score `ocr_score` = 0.7.

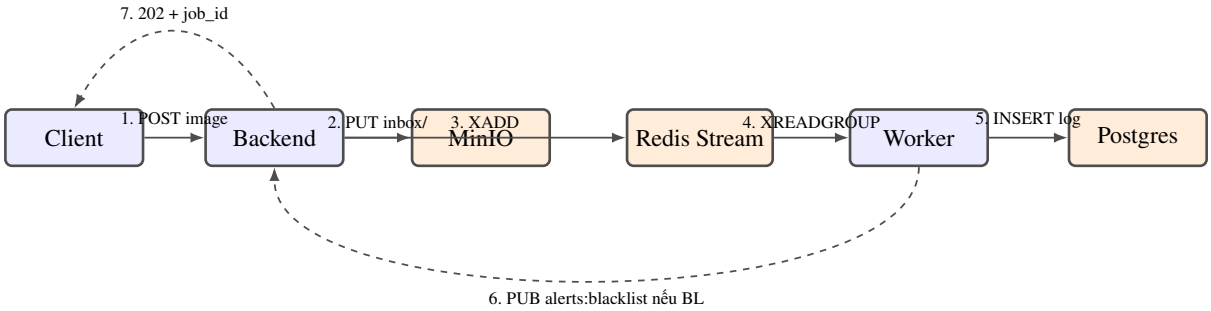
Mỗi crop SUCCESS được đẩy thêm qua ResNet18 (`fc=Identity`, L2-norm) để sinh vector 512-d và upsert vào Qdrant, phục vụ tìm xe tương đồng theo ảnh.

4.2 Cơ sở dữ liệu

Schema PostgreSQL gồm 6 bảng chính: `users`, `cameras`, `vehicles`, `watchlists`, `recognition_logs`, `refresh_tokens` và bảng audit `system_logs`. Một số index đáng chú ý:

- `idx_recognition_plate_trgm` - GIN trigram trên `plate_text` cho phép tìm gần đúng (`51A123` match `51A-123.45`).
- `idx_watchlists_blacklist` - partial index riêng cho BLACKLIST, tăng tốc kiểm tra alert.
- `idx_recognition_unverified` - partial cho log `SUCCESS` chưa verify, giúp trang admin review nhanh.
- `idx_system_logs_detail` - GIN trên cột JSONB phục vụ truy vết.

4.3 Luồng xử lý nhận diện không đồng bộ



Hình 3: Luồng nhận diện không đồng bộ qua Redis Stream

4.4 API tóm tắt

Tổng cộng $\sim$ 30 endpoint, phân nhóm: `/auth/*`, `/users/*`, `/cameras/*`, `/vehicles/*`, `/watchlists/*`, `/recognition/*`, `/events/alerts` (SSE), `/streams/cameras/{id}` (MJPEG), `/stats/*`. Tài liệu sinh tự động bằng Swagger tại `http://localhost/d`

4.5 Quản lý phiên bản model & dữ liệu

Một điểm cộng quan trọng: hệ thống tổ chức theo chuẩn MLOps thu nhỏ gồm `params.yaml` (hyperparams), `dvc.yaml` (4 stage: `train_detector`, `eval_detector`, `train_ocr`, `eval_ocr`), `MODELS.md` (model registry) và thư mục `metrics/` chứa metrics đầu ra (DVC track). Model weight được push lên MinIO (`s3://dvc-store`). Sau khi sửa hyperparam, chỉ cần `dvc repro` là DVC tự rerun đúng stage bị ảnh hưởng; `dvc metrics diff` cho phép so sánh giữa các version model.

5 Triển khai

5.1 Khởi động nhanh

Image backend và frontend đã pre-build trên Docker Hub (`dddddd112233/lpr-backend:1.0` và `dddddd112233/lpr-frontend:1.0`). Người chấm chỉ cần:

```
git clone <repo-url> OwnerCheckAI
cd OwnerCheckAI
docker compose up -d
```

Compose tự pull image, không cần `.env`, không cần `dvc pull`, không cần build local. Lần pull đầu tiên khoảng 5–10 phút (tổng dung lượng nén 2 GB). Mở UI tại `http://localhost/`, Swagger tại `http://localhost/docs`.

User đăng ký đầu tiên qua `POST /auth/register` sẽ được auto-set role `ADMIN` (bootstrap); từ user thứ hai trở đi luôn là role `USER`.

5.2 Workflow developer

Khi sửa code và muốn build image local thì cần thêm bước kéo weight qua DVC:

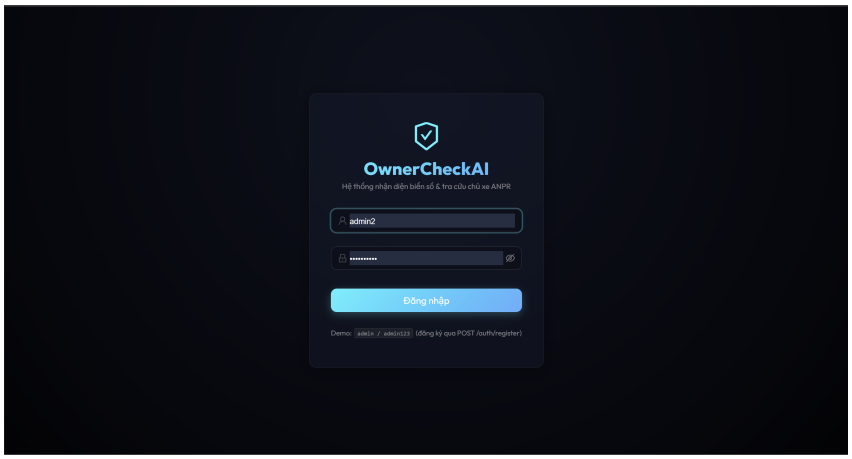
```
docker compose up -d minio minio-setup
dvc remote modify --local minio access_key_id    "$MINIO_ROOT_USER"
dvc remote modify --local minio secret_access_key "
   $MINIO_ROOT_PASSWORD"
dvc pull
docker compose build
docker compose up -d
```

6 Giao diện người dùng

Phần này minh họa các màn hình chính của frontend. Toàn bộ giao diện xây dựng bằng React 18 + Ant Design 5; menu trái thay đổi theo role (User vs Admin).

6.1 Đăng nhập

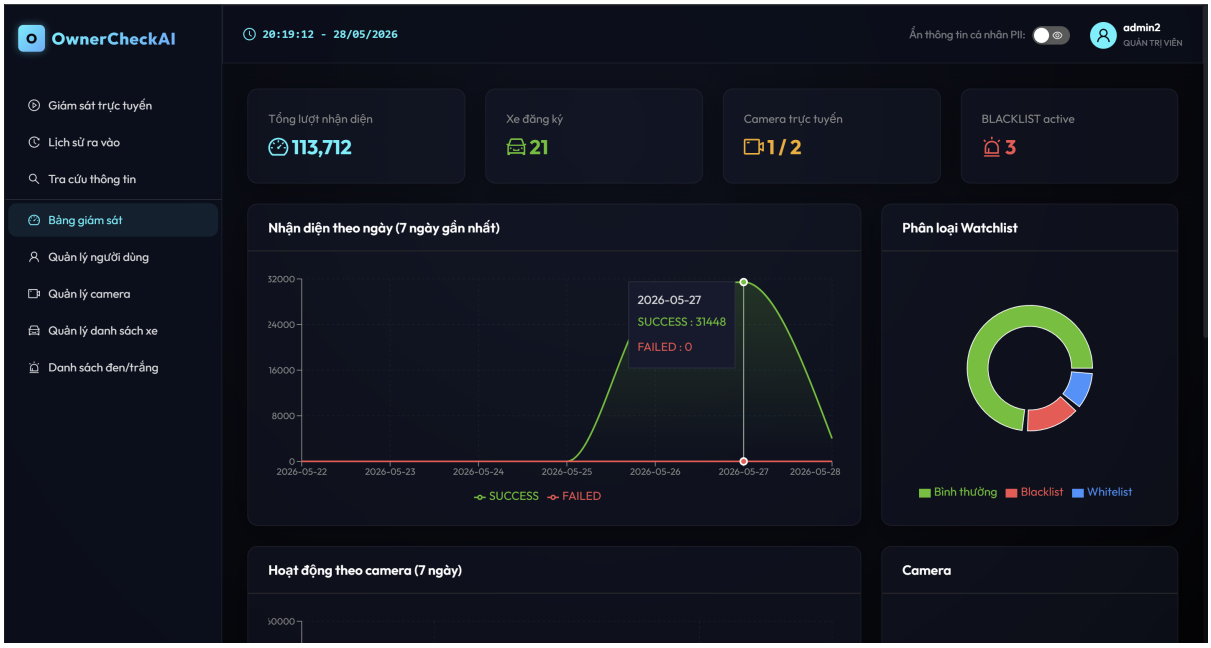
Trang đăng nhập sử dụng JWT (access + refresh token). Người dùng đầu tiên đăng ký qua `POST /auth/register` được tự động gán role `ADMIN`; từ user thứ hai trở đi luôn là role `USER` (bootstrap an toàn, ngăn leo quyền).



Hình 4: Trang đăng nhập

6.2 Bảng giám sát (Dashboard) – dành cho Admin

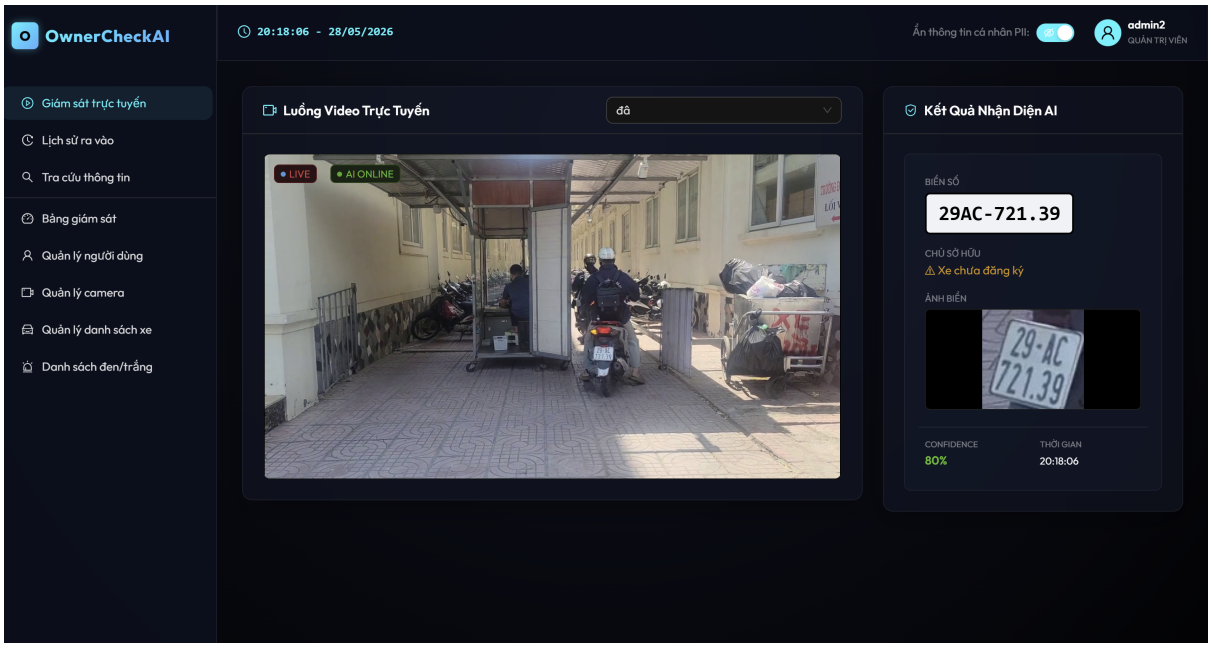
Hiển thị 4 KPI ở đầu trang (tổng lượt nhận diện, số xe đã đăng ký, camera đang trực tuyến, số mục BLACKLIST đang active) và 3 biểu đồ chính: số log SUCCESS/FAILED theo ngày, hoạt động từng camera, tỷ lệ các loại watchlist.



Hình 5: Dashboard: KPI + biểu đồ nhận diện 7 ngày + phân loại watchlist

6.3 Giám sát trực tuyến (Live view)

Luồng MJPEG realtime của camera được chọn (10 fps) hiển thị bên trái; panel *Kết quả nhận diện AI* bên phải cập nhật theo từng kết quả AI mới nhất: biển số, ảnh crop, ngưỡng confidence và trạng thái đối chiếu chủ xe (đã đăng ký / chưa đăng ký / BLACKLIST).

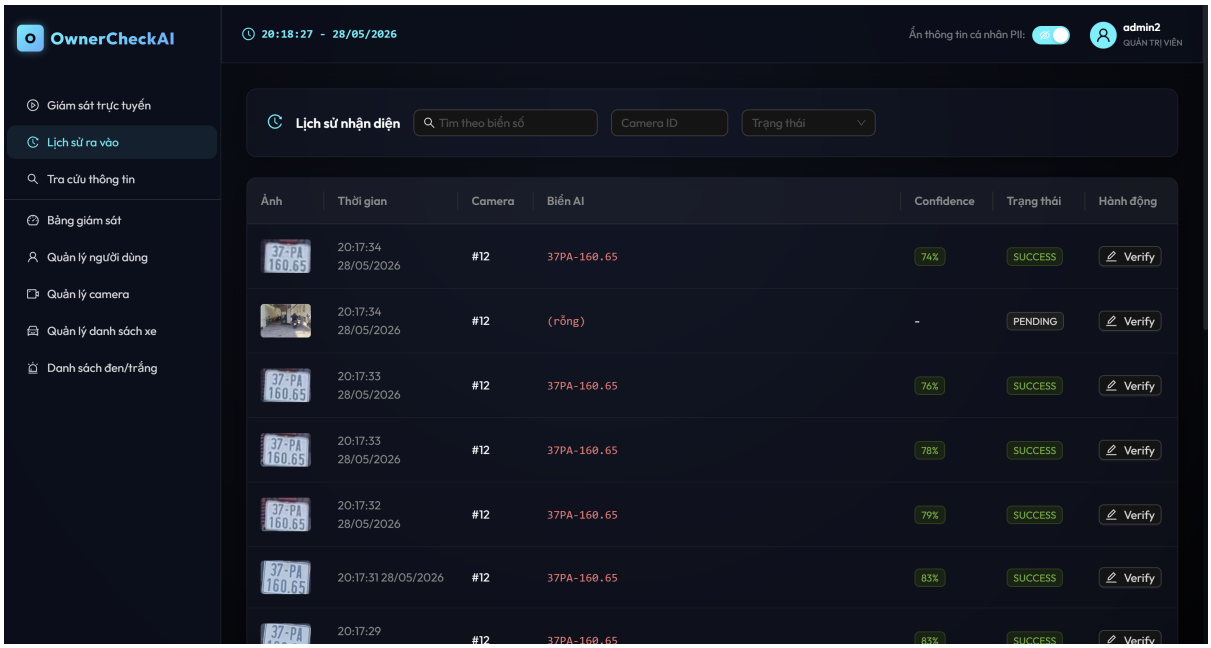


Hình 6: Live view: luồng MJPEG + panel kết quả nhận diện AI realtime

6.4 Lịch sử ra vào (History logs)

Toàn bộ log nhận diện được lưu vào bảng `recognition_logs` kèm ảnh crop biển số lưu trên MinIO. Trang này cho phép filter (biển số, camera, trạng thái), xem ảnh crop và *manual*

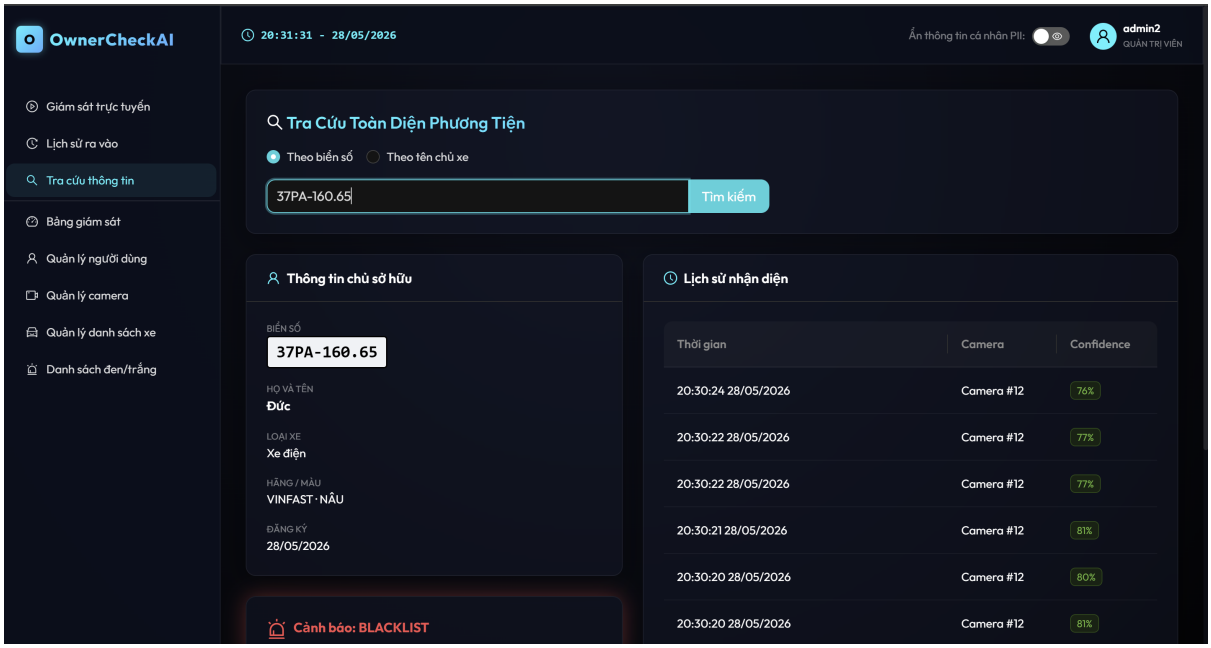
verify khi OCR sai (status chuyển từ `PENDING` sang `SUCCESS` hoặc `FAILED`).



Hình 7: Lịch sử nhận diện: filter + ảnh crop + confidence + manual verify

6.5 Tra cứu phương tiện

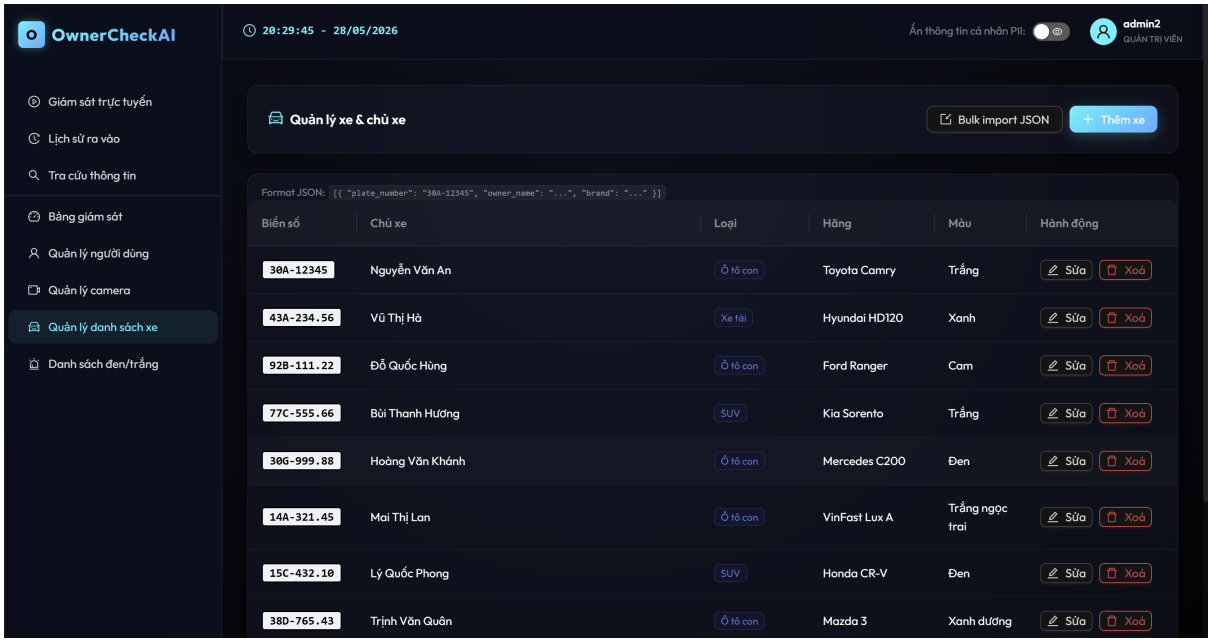
Tra cứu theo biển số hoặc theo tên chủ xe (dùng index trigram `pg_trgm` nên match được biển viết tay khác dấu gạch). Khi tìm thấy, hệ thống hiển thị đồng thời *thông tin chủ sở hữu*, *trạng thái watchlist* (BLACKLIST/WHITELIST nếu có) và toàn bộ *lịch sử nhận diện* của xe đó.



Hình 8: Trang tra cứu: card chủ xe + lịch sử nhận diện + cảnh báo BLACKLIST

6.6 Quản lý xe & bulk-import (Admin)

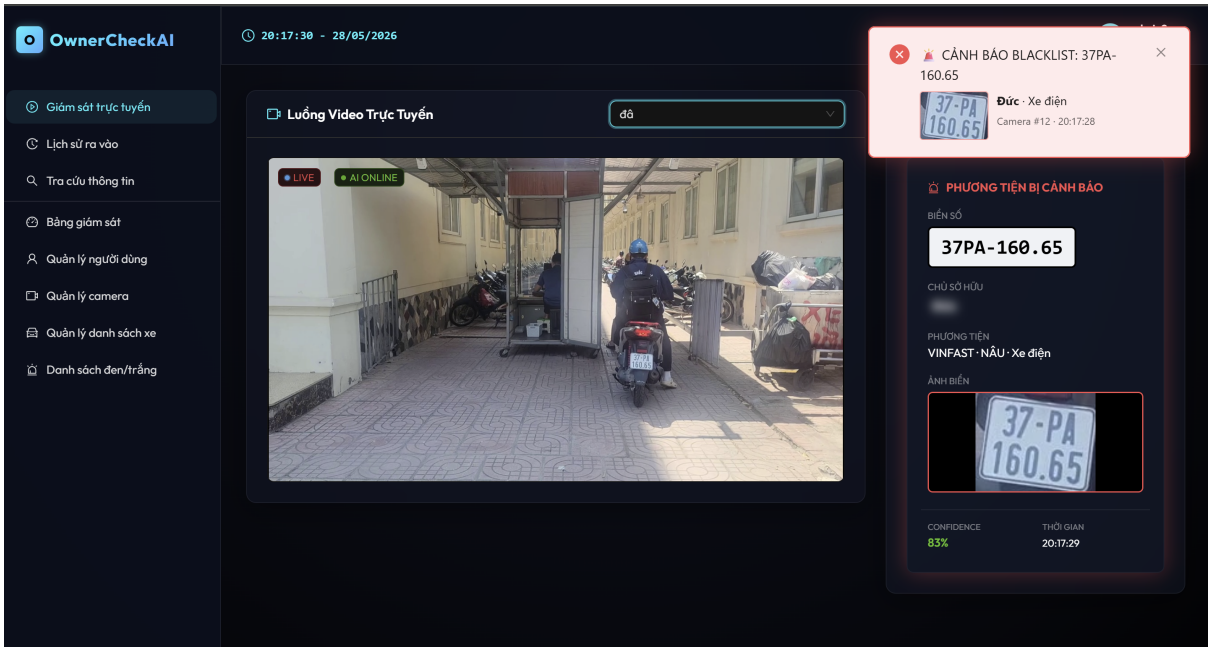
Trang quản lý xe hỗ trợ CRUD từng bản ghi và *bulk-import JSON* tối đa 1000 dòng / request - hữu ích khi cần nạp danh sách xe từ hệ thống sẵn có.



Hình 9: Quản lý xe: CRUD biển số, chủ xe, hãng, loại, màu + bulk-import JSON

6.7 Cảnh báo BLACKLIST realtime

Khi worker phát hiện một biển số thuộc BLACKLIST, nó `PUBLISH` event `alerts:blacklist` qua Redis Pub/Sub. Backend đang `SUBSCRIBE` kênh này sẽ đẩy event `blacklist_match` qua Server-Sent Events tới mọi tab admin đang mở. Frontend hiển thị toast cảnh báo ngay lập tức, kèm panel chi tiết biển số - chủ xe - ảnh crop - camera nguồn.



Hình 10: Toast cảnh báo BLACKLIST realtime đẩy qua SSE - hiển thị ngay trên Live view kèm panel chi tiết phương tiện

7 Phân công công việc

Trong suốt quá trình phát triển, nhóm chia thành 3 mảng tương ứng với ba vai trò chính: Backend/DevOps, AI Engine và Frontend. Mỗi thành viên chịu trách nhiệm chính một mảng nhưng vẫn cộng tác chéo trong các phần liên kết (ví dụ định nghĩa schema API, format payload SSE, contract của queue).

7.1 Bảng phân công chi tiết

Hạng mục công việc	Thành viên phụ trách	Tỷ lệ
Thiết kế CSDL, schema, index trigram	Lê Mạnh Cường	100%
Backend FastAPI (auth, CRUD, validation)	Lê Mạnh Cường	100%
Worker AI + Redis Stream consumer	Lê Mạnh Cường	100%
Stream-ingestor (RTSP → frames, MJPEG, SSE)	Lê Mạnh Cường	100%
Docker Compose, nginx, MinIO, Qdrant	Lê Mạnh Cường	100%
Build & push image lên Docker Hub	Lê Mạnh Cường	100%
Huấn luyện YOLOv8m phát hiện biển VN	Trần Tuấn Kiệt	100%
Fine-tune PaddleOCR PP-OCRv3	Trần Tuấn Kiệt	100%
LPR pipeline (preprocess + integrate)	Trần Tuấn Kiệt	100%
Embedder ResNet18 + Qdrant integration	Trần Tuấn Kiệt	100%
DVC pipeline + `params.yaml` + `MODELS.md`	Trần Tuấn Kiệt	100%
Thiết kế UI (Ant Design 5)	Nguyễn Minh Đức	100%
Frontend User (Live, History, Lookup)	Nguyễn Minh Đức	100%
Frontend Admin (CRUD + dashboard + alert SSE)	Nguyễn Minh Đức	100%
Tài liệu README & báo cáo	Nguyễn Minh Đức	100%

Bảng 3: Bảng phân công chi tiết các hạng mục

7.2 Tỷ lệ đóng góp tổng thể

MSSV	Họ tên	Vai trò chính	Tỷ lệ đóng góp
24022276	Lê Mạnh Cường	Backend / DevOps	33,33%
24022376	Trần Tuấn Kiệt	AI Engine + Model versioning	33,33%
24022294	Nguyễn Minh Đức	Frontend + Tài liệu	33,34%

Bảng 4: Tỷ lệ đóng góp của các thành viên

8 Kết luận

Đồ án đã xây dựng thành công hệ thống nhận diện biển số xe thời gian thực *OwnerCheckAI* chạy đầy đủ end-to-end:

- Đáp ứng đầy đủ 7/7 thành phần bắt buộc trong rubric: frontend (User + Admin), backend FastAPI, AI worker, 3 storage (Postgres, MinIO, Qdrant), queue Redis Stream, load balancer nginx.
- Có thêm các *điểm cộng*: Docker Compose tích hợp toàn bộ stack, Redis LRU cache đa lớp, vector DB cho similarity search, quản lý phiên bản model & data bằng DVC, RTSP simulator (MediaMTX) cho phép demo không cần camera vật lý.
- Triển khai chỉ với 1 lệnh `docker compose up -d` - dễ tái chấm và mở rộng.

Hướng phát triển: Tích hợp CI/CD trên GitHub Actions để tự động build/push image; triển khai Kubernetes + Helm cho môi trường production; thêm monitoring Prometheus + Grafana; backup tự động cho Postgres và MinIO; deploy ra Internet kèm chứng chỉ SSL (Let's Encrypt).